Graph Convolutional Network With Local and Global Feature Fusion for Hyperspectral Image Classification

Yufan Wang, Xiaodong Yu, Hongbin Dong, and Shuying Zang

***Abstract*— In hyperspectral image (HSI) classification tasks, extracting surface features is crucial, but the complexity of HSI brings considerable challenges. Traditional convolutional neural networks (CNNs) and vision transformer (ViT) perform well but have large parameters and are time-consuming. In contrast, graph convolutional neural networks (GCNs) are fast to train with few parameters. The simple linear iterative clustering (SLIC) segmentation algorithm generates pixel-level graph data and superpixel-level graph data. Single-branch GCNs learn only one type of graph data resulting in insufficient feature richness. In two-branch GCNs, one branch extracts local spatial–spectral features from pixel-level data, another branch extracts global spatial–spectral features from superpixel-level data, and finally, the two branches are combined. However, the existing two-branch GCNs do not focus on the close fusion of local and global features. Therefore, in this article, we propose new two-branch GCNs with local and global feature fusion (LG-GCNs). The LG-GCN model proposed in this study not only has a short training time but also has a high accuracy. The LG-GCN can capture more comprehensively the local and global spatial–spectral semantic information of HSI. The experimental results on three publicly available HSI datasets show that LG-GCN outperforms other state-of-the-art models in terms of classification accuracy and robustness. In addition, it has a shorter training time and is more resistant to noise. The source code of the LG-GCN model is available at https://github.com/Dreamvai/LG-GCN.**

***Index Terms*— Graph convolutional neural network (GCN), hyperspectral imaging (HSI) classification.**

I. Introduction

BECAUSE each substance has unique spectral properties, remote sensing detectors are able to record the spectral distribution of light reflected back from each substance, which makes it possible to use hyperspectral images (HSIs) to identify objects on the ground. Therefore, in recent years, hyperspectral imagery has been used in many fields such as fine agriculture [1], urban planning [2], accurate agriculture [3], [4], environmental management [5], and land cover change detection [6]. However, the high dimensionality of hyperspectral images in the spectrum makes it difficult to achieve high-precision classification.

At an earlier time, machine learning methods were popular for hyperspectral image classification tasks, such as support vector machines (SVMs) [7], random forests (RFs) [8], Markov random fields [9], and K-nearest neighbor (KNN) [10] algorithm. Traditional machine learning methods usually rely on manual design, which requires extensive experience and expertise, and when the data are very complex, their fitting ability is greatly reduced. Later, Chen et al. [11] first introduced deep learning to HSI classification. Deep learning automatically learns high-level semantic features of the data through a multilayer network structure and is often better able to capture and represent the complex structure and semantics of the data. After the great success of convolutional neural networks (CNNs) on other image classification tasks, many CNN-based methods are applied to hyperspectral image classification. One-dimensional-CNN [11], [12], [13] is proposed. However, 1D-CNN focuses more on the spectral information of individual pixels and ignores the spatial information. The emergence of 3D-CNN [14], [15], [16], [17], [18] realizes the fusion of spatial and spectral information of HSI. In addition, Feng et al. [19] also considered the class imbalance of the dataset when designing the network. However, the CNN requires many training parameters and is prone to overfitting with a limited number of training samples. In addition, CNNs rely excessively on local receptive fields and are not good at capturing global features.

Meanwhile, after Dosovitskiy et al. [20] proposed the vision transformer (ViT), ViT-based models caused a sensation in the image classification field. Hong et al. [21] explored the performance of ViT on the HSI classification task. Later, a number of networks combining ViT and convolution appeared, such as SSFTT [22], SSTN [23], and CTMixer [24]. These methods maximize the advantages of convolution and ViT and combine them perfectly. They try to extract local features by convolution and global features by transform encoder. Different types of features are obtained

Received 22 August 2024; revised 26 September 2024; accepted 25 October 2024. Date of publication 28 October 2024; date of current version 12 November 2024. This work was supported in part by the National Natural Science Foundation of China under Grant U20A2082 and Grant 41971151 and in part by Harbin Normal University Postgraduate Innovation Project under Grant HSDSSCX2022-117. *(Corresponding author: Xiaodong Yu.)*

Yufan Wang and Xiaodong Yu are with the Faculty of Computer Science and Information Engineering, Harbin Normal University, Harbin 150025, China (e-mail: wangyufan1415@gmail.com; yuxiaodong@hrbnu.edu.cn).

Hongbin Dong is with the Faculty of Computer Science and Technology, Harbin Engineering University, Harbin 150001, China (e-mail: donghongbin@hrbeu.edu.cn).

Shuying Zang is with the Key Laboratory of Geographical Environment Monitoring and Spatial Information Service in Cold Regions of Heilongjiang Province, Harbin Normal University, Harbin 150025, China (e-mail: zsy6311@163.com).

Digital Object Identifier 10.1109/TGRS.2024.3487296

1558-0644 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 13:24:16 UTC from IEEE Xplore. Restrictions apply.

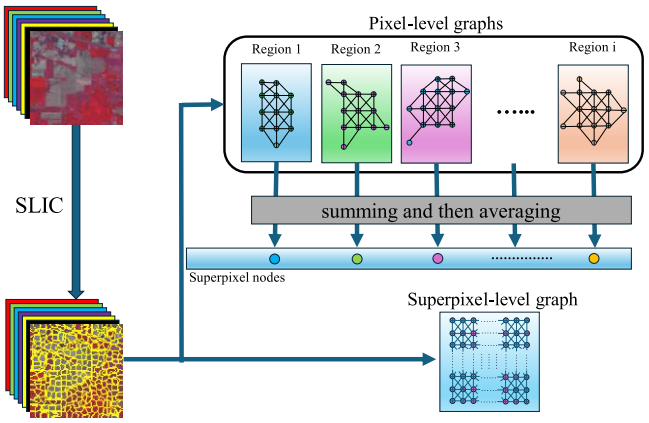


Fig. 1. Process of generating graph-structured data using images after SLIC segmentation. A pixel-level graph is composed of all pixel nodes inside a superpixel region. The superpixel-level graph is composed of all superpixel nodes. All the pixel node feature vectors in superpixel i are summed and averaged to obtain the feature vector of superpixel node i. To find out how to determine the edges between nodes, refer to Algorithm 1.

through convolution and transformer encoder and then fused to capture spectral–spatial features and high-level semantic information. From the previous work, in order to achieve high classification accuracy on HSI classification tasks, both CNN- and ViT-based methods need to focus on extracting spatial–spectral features and local–global features.

To deal with handling irregularly structured data, graph convolutional networks (GCNs) [25], [26] have been proposed by researchers. Graph convolutional networks are capable of accomplishing classification tasks in a semi-supervised manner, requiring only a small number of samples. What is more, GCN aggregates feature information of neighboring nodes at each node so that not only local features but also global features can be captured through multilayer passes. This message passing mechanism enables the GCN to capture complex structures and relationships between nodes in the graph. For better extraction of local–global and spectral–spatial features, GCNs has also been introduced for HSI classification [27], [28], [29], [30], [31]. Qin et al. [27] proposed S2GCN, which constructs the graph by searching KNNs for each pixel through the KNN algorithm. However, this method of constructing pixel-level graphs is particularly cumbersome. In order to solve this problem, it has been proposed to construct a superpixel-level graph by splitting the HSI into many superpixels using an oversegmentation algorithm, e.g., the simple linear iterative clustering (SLIC) [32] algorithm. Fig. 1 and Algorithm 1 show the process of obtaining pixel-level graph data and superpixel-level graph data by the SLIC segmentation algorithm. Then, GCNs were used to update the features of the superpixel-level graph. However, considering only the features of this superpixel-level graph leads to the lack of information within this superpixel region. Therefore, some new GCN-based methods [33], [34], [35] are proposed that consider both the features of pixel-level graphs as well as superpixel-level graphs. Liu et al. [33] proposed CEGCN, which extracts spectral features in pixel-level and superpixel-level spaces by combining the advantages of CNN and GCN together. Yang et al. [34] proposed a new pixel-level graph node updating (PGNU) strategy to update pixel- and superpixel-level

Algorithm 1 Establishing the Topology of Graph

```
Input [pixel, superpixel] = Picture segmented by SLIC
### For pixel-level graphs
for pixel-A in Input[pixel]:
    pixels = [other pixels adjacent to this pixel-A]
        for pixel-B in pixels:
            if pixel-A and pixel-B do not belong to same
            superpixel region:
                Continue
            if the edge connecting pixel-A and pixel-B
            exists:
                Continue
            else:
                create an edge connecting pixel-A
                and pixel-B
Obtain pixel-level graphs
### For superpixel-level graph
for superpixel-A in Input[superpixel]:
    superpixels = [other superpixels adjacent to
    this superpixel-A]
        for superpixel-B in pixels:
            if the edge connecting superpixel-A
            and superpixel-B exists:
                Continue
            else:
                create an edge connecting superpixel-A
                and superpixel-B
Obtain superpixel -level graph
```



Fig. 2. Framework based on GCNs.

features simultaneously. Jia et al. [35] proposed to extract the features of the internal and external graphs by two independent graph convolutions. In addition, GTFN [36] combining ViT and GCN has been proposed, where they want to combine the long-term dependency modeling capability of transformer with the node modeling capability of GCN.

The GCN methods mentioned above can be divided into four groups. The first group of GCNs methods is to first process the raw data structure into a kind of trainable data, then extract the features through graph convolutional layers, and finally make linear classification, as shown in Fig. 2. The representative methods of this group include S2GCN [27], S^2GraphSAGE [29], and SSAPGCN [30].

The second group is based on the framework of the first one and integrates convolution or ViT into it. Namely, some of the GCN layers in Fig. 2 are replaced with convolutional or ViT layers. Representative methods of this group include GTFN [36]. The third group is to first process the raw data into one or two trainable data, then extract the features by two or more branches of graph convolutional layers, and finally make a linear classification, as shown in Fig. 3. The representative methods in this group include MDGCN [28], DMSGer [34], and GiGCN [35].

The fourth group is based on the framework of the third one and integrates convolution or ViT into it, that is, some of the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 13:24:16 UTC from IEEE Xplore. Restrictions apply.

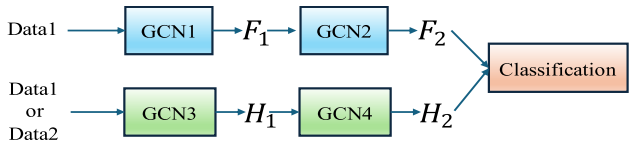


Fig. 3. Another GCN-based framework.

GCN layers in Fig. 3 are replaced with convolutional or ViT layers. Representative methods of this group are CEGCN [33].

Although the above GCN-based methods have achieved good results, they all have some limitations. First, the first two groups of methods extract only one kind of feature from trainable data, so there is the problem of insufficient feature richness. Second, the latter two groups of methods emphasize the importance of pixel-level features and superpixel-level feature extraction with some feature richness, but the fusion of these two features is not strong enough; they are only fused at the time of the linear classification to be performed, and there is no connection between different branches. Furthermore, we believe that GCN is fully capable of extracting spatial–spectral and local–global features of HSI through a message passing mechanism. There is no need to combine with CNN or transformer. In other words, GCNs can be rationally designed to enable local feature extraction and explore long-range dependencies.

Therefore, in this article, we design a network architecture consisting entirely of graph convolution, focusing on local–global feature fusion. The model can be split into three parts, i.e., the GCN branch, the simplifying graph convolutional network (SGCN) branch, and the feature fusion operation; the GCN branch is used to process the pixel-level graphical data to extract the spectral and local spatial features of the pixels, while the SGCN branch is used to process the superpixel-level graphical data to extract the spectral and global spatial features. The feature fusion operation is a multilevel fusion of the features extracted from the two branches. Fig. 4 shows the detailed network architecture. The main contributions of this article are summarized as follows.

1) In this article, we design a novel end-to-end semi-supervised network that both extracts superpixel-level features and performs pixel-level feature extraction within each superpixel region. In addition, we suggest using SGCN [37] to capture a large range of global features on the superpixel-level graph without missing important details, thus improving the overall feature extraction capability and classification accuracy.
2) In our research, we focus on how to more effectively combine superpixel- and pixel-level features to achieve a deep fusion of features at both local and global levels. Our proposed GCN with local and global feature fusion (LG-GCN) model, through a multistage feature fusion process, is able to extract richer local and global information, thereby significantly enhancing the performance of HSI classification tasks. This multilevel fusion strategy not only strengthens the model's ability to capture HSI details but also improves the quality of the overall feature representation.
3) Finally, due to the inherent complexity and high dimension of HSI data, we have introduced an ensemble learning strategy to enhance our LG-GCN network. This approach, by combining the predictive results of multiple models, improves overall performance and reduces the bias of the predictions, thereby improving the robustness. Our research demonstrates that ensemble learning is effective in enhancing the ability of graph neural networks to analyze complex graph-structured data, resulting in higher stability and accuracy of the model when dealing with complex data.

To validate the efficiency and stability of our proposed LG-GCN model in HSI classification tasks, we have selected three widely used public datasets as benchmarks. We considered not only the classification accuracy of the model but also focused on the time efficiency required for data processing. Through detailed comparative analysis with the leading CNN-, ViT-, and GCN-based classification models, our LG-GCN has demonstrated outstanding performance. Furthermore, to demonstrate the excellent noise immunity of the LG-GCN model and the advantages of ensemble learning, we have also conducted tests on noise immunity and demonstrated the ensemble learning effect of models learned under different noises. Finally, we have publicly released the source code of the LG-GCN model and other methods used for comparison at https://github.com/Dreamvai/LG-GCN.

II. Proposed Method

In this section, we will detail the various techniques and methods employed in the LG-GCN model. Initially, we apply the SLIC algorithm to segment the HSI and use this segmented image as the foundation for constructing graph-structured data. Fig. 1 and Algorithm 1 clearly illustrate the process of generating graph-structured data. Subsequently, we extract local and global features of the HSI through the graph convolution process. Fig. 4 displays the overall design architecture of the LG-GCN model. In the final stage of the model, we introduce ensemble learning techniques to enhance the model's performance and stability, ensuring more reliable results. The integrated application of these methods enables the LG-GCN model to perform excellently in HSI classification tasks.

A. Simple Linear Iterative Clustering

The SLIC algorithm was proposed by Achanta et al. [32] for image segmentation to generate superpixels. The superpixels created by the SLIC algorithm are indeed cell-like in their compactness and orderly arrangement, which simplifies the expression of neighborhood features. More importantly, methods originally based on pixels can be relatively easily transformed into superpixel-based approaches, enhancing the efficiency and effectiveness of image processing.

The SLIC algorithm is akin to the K-means algorithm. The elements in the SLIC algorithm are briefly described next. Suppose that an image has a total of N pixels and is intended to be segmented into K superpixels, and there will be K

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 13:24:16 UTC from IEEE Xplore. Restrictions apply.

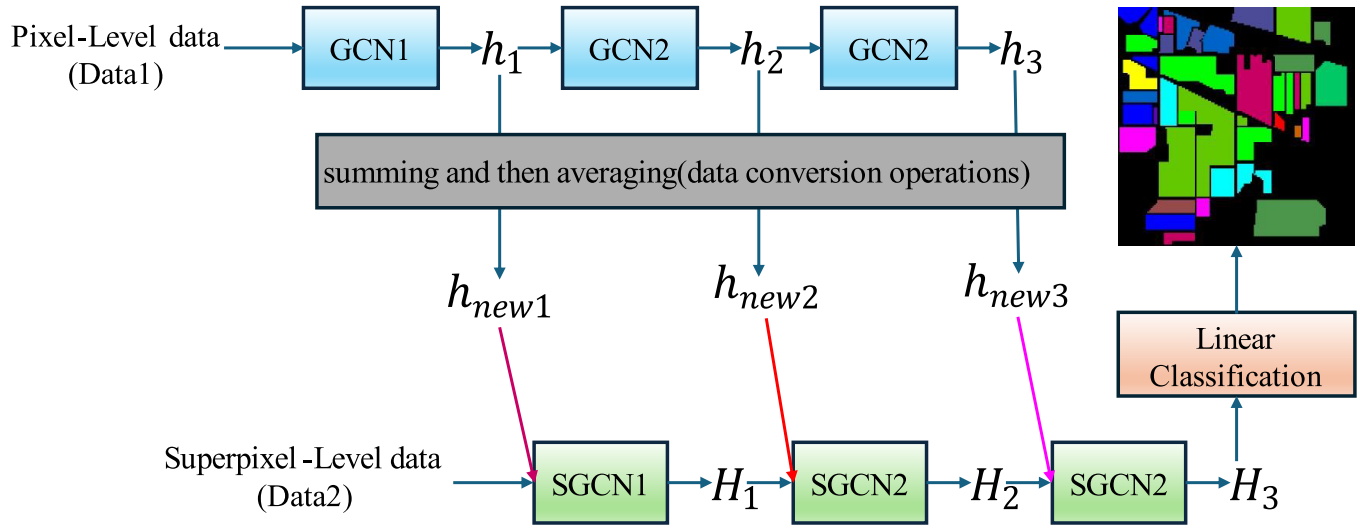


Fig. 4. Overall architecture of LG-GCN.

seed points. The step size S between seeds is calculated as

$$S = \sqrt{\frac{N}{K}}. \tag{1}$$

A cluster is then formed by assigning the nearest seed point to each pixel in a $2S \times 2S$ region around each seed point. The distance D is calculated by the following formula:

$$d_{\mathrm{RGB}} = \sqrt{(R_p - R_s)^2 + (G_p - G_s)^2 + (B_p - B_s)^2} \tag{2}$$

$$d_{xy} = \sqrt{(x_p - x_s)^2 + (y_p - y_s)^2} \tag{3}$$

$$D = \sqrt{\left(\frac{d_{\mathrm{RGB}}}{m}\right)^2 + \left(\frac{d_{xy}}{S}\right)^2} \tag{4}$$

where d_{RGB} denotes the spectral distance from pixel point i to seed point s and d_{xy} denotes the spatial distance from pixel point p to seed point s. S is obtained from (1), and m is the weight value. Moreover, the SLIC algorithm employs an operation to enhance connectivity, addressing issues of superpixels being too small or discontinuous.

In our experiments, by indirectly adjusting the K value, we can generate segmented images with different characteristics. This K value is crucial for determining the number of superpixels, which directly affects the performance of the LG-GCN model in HSI classification tasks. When the K value increases, the number of superpixels increases, but the number of pixels within each superpixel decreases, which may lead to insufficient spatial feature information within the superpixel and affect the model's performance. Conversely, when the K value decreases, the number of superpixels decreases, but the number of pixels within each superpixel increases, which may result in a lack of sufficient global feature information. Therefore, in practical applications, finding an appropriate K value to balance the extraction of local and global features is essential for improving the classification accuracy and robustness of the LG-GCN model. Our experimental results also confirm this, as optimizing the choice of K value has enabled our LG-GCN model to achieve better classification results on different datasets.

B. GCN and SGCN

Suppose that there exists an undirected graph $G = (V, E, A)$ with N nodes, V is all the nodes in the graph G, and E is all the edges in the graph G that connect two nodes. $v_i \in V$ is denoted as a node and $e_{ij} \in E$ is denoted as an edge connecting v_i and v_j. A node in the graph is a vector (vector dimension is d), and the vectors of all the nodes in the graph are saved as the matrix $X \in R^{N\times d}$. $x_i \in X$ is denoted as the vector of v_i. $A \in R^{N\times N}$ is the adjacency matrix of the graph G and the value of each element of A is defined as follows:

$$A_{ij} = \begin{cases} 1, & \text{if } e_{ij} \in E \\ 0, & \text{if } e_{ij} \notin E. \end{cases} \tag{5}$$

The degree matrix D is a diagonal matrix and can be computed from the adjacency matrix A, where $D_{ij} = \sum_j A_{ij}$. Then, the Laplace matrix of the graph G is

$$L \in R^{N\times N} = D - A. \tag{6}$$

After the Laplacian matrix is normalized

$$L_n \in R^{N\times N} = I - D^{-\frac{1}{2}} A D^{-\frac{1}{2}} \tag{7}$$

where I is an identity matrix. Because L_n is a real symmetric semi-positive definite matrix, it has N eigenvectors $u_1, \ldots, u_n$ ($|u_i| = 1$) and N eigenvalues $\lambda_1, \ldots, \lambda_n$. $U \in R^{N\times N} = [u_1, \ldots, u_n]$ and $\wedge \in R^{N\times N} = \mathrm{diag}(\lambda_1, \ldots, \lambda_n)$. The following equations hold:

$$L_n = U \wedge U^{\mathrm{T}}. \tag{8}$$

Then, $g_\theta = \mathrm{diag}(\theta \in R^N)$ was introduced to simulate convolution operations and can be viewed as a filter for graph convolution. The graph convolution process can be expressed as

$$g_\theta * X = U g_\theta(\wedge) U^{\mathrm{T}} X. \tag{9}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 13:24:16 UTC from IEEE Xplore. Restrictions apply.

Since both $\wedge$ and g_θ are diagonal matrices, g_θ in (9) can be viewed as the function mapping of the diagonal matrix. To reduce the computational effort of feature decomposition, a Chebyshev polynomial is used to fit $g_\theta(\wedge)$

$$g_\theta(\wedge) \approx \sum_{k=0}^{K} \theta_k T_k(L_n) \tag{10}$$

where T_k is the Chebyshev polynomial and θ_k denotes a vector of Chebyshev coefficients. K in (10) is set to 1, which means that the first-order Chebyshev polynomials are taken. Equation (9) is simplified as follows:

$$g_\theta * X = \theta\left(\tilde{D}^{-\frac{1}{2}}\tilde{A}\tilde{D}^{-\frac{1}{2}}\right)X \tag{11}$$

where $\tilde{A} = A + I$ and $\widetilde{D_{ij}} = \sum_j \widetilde{A_{ij}}$.

Finally, the activation is added to obtain the propagation rule for the GCN [25]

$$H^{(l+1)} = \sigma\left(\tilde{D}^{-\frac{1}{2}}\tilde{A}\tilde{D}^{-\frac{1}{2}}H^{(l)}W^{(l)}\right) \tag{12}$$

where $H^{(l)}$ is the feature matrix of the input of the lth layer, and then, $H^{(l+1)}$ is the feature matrix of the output of the lth layer and the input of the $(l+1)$th layer. σ is the activation function and $W^{(l)}$ is the learnable parameter at the lth layer.

Wu et al. [37] hypothesized that the nonlinear activation functions between GCN layers are not important at all, and the key is the local averaging of feature propagation. Therefore, SGCN is proposed, which removes the nonlinear activation function between the layers. For convenience, it is assumed that $Q = \tilde{D}^{-\frac{1}{2}}\tilde{A}\tilde{D}^{-\frac{1}{2}}$. In SGCN, (12) is adapted to

$$H^{(l+1)} = \sigma\left(Q^m H^{(l)} W^m\right). \tag{13}$$

To easily explain the difference between (12) and (13), assume that $m = 2$. If the nonlinear activation function between the layers is preserved in SGCN, (13) changes to

$$H^{(l+1)} = \sigma\left(Q\left(\sigma\left(QH^{(l)}W\right)\right)W\right). \tag{14}$$

Equation (12) shows that GCN processes only the features of first-order neighbor nodes in each iteration. In contrast, (14) indicates that SGCN can process the features of both first- and second-order neighbor nodes in a single iteration. This means that a single layer of SGCCN (when m equals 2) is equivalent to the effect of two layers of GCN. If the nonlinear activation function between the layers is removed in SGCN, (14) changes to

$$H^{(l+1)} = \sigma\left(QQH^{(l)}WW\right). \tag{15}$$

Equation (15) is the special case of (13) when m equals 2. Since SGCN can aggregate graph information comparable to that of a multilayer GCN, it possesses the same range of receptive field as a multilayer GCN. This characteristic allows SGCN to capture the global spatial features when processing graph-structured data, thereby enhancing the model's performance more comprehensively.

As shown in Fig. 5, the SLIC algorithm produces many superpixels after segmentation, but the number of pixels inside each superpixel is small. In this case, when we use SGCN to update pixel-level features, we set $m = 1$, which makes SGCN equivalent to GCN. This means that each node only aggregates information from its direct neighbors. Due to the large number of superpixels and the fact that neighboring superpixels are likely to belong to the same label, we set $m = 2$ when updating the superpixel-level features. This allows each node to aggregate information from its more distant neighbors, which increases the width of the receptive domain. The SGC acts as a low-pass filter that helps to produce smooth features on the graph. This means that the feature representations of neighboring nodes become similar, which results in them having similar predictions. This property is useful for tasks such as image segmentation and object classification, as it allows us to exploit information from localized superpixels to make predictions. However, the value of m should not be too large as it can lead to oversmoothing, i.e., nodes with different labels may incorrectly obtain similar representations, which reduces the accuracy of the classification. In addition, too large a value of m may also lead to model overfitting; as the number of model layers increases, the model may overfit to the noise in the training data instead of learning the true distribution of the data.

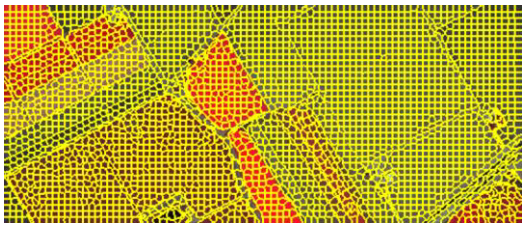


Fig. 5. Image segmented by SLIC.

C. Ensemble Learning

In our ensemble learning experiments, we utilized different hyperparameters of the SLIC algorithm to segment the HSI, which served as the foundation for constructing graph-structured data. This approach resulted in graph-structured data with diversity. By learning from these varied graph-structured data through the LG-GCN model, the model can focus on different features. For the final predictions, we combined the prediction results from multiple LG-GCN models and used a majority voting decision to determine the ultimate prediction outcome. To obtain segmented images with significant differences in the ensemble learning experiments, we controlled the step size S between superpixels. However, the hyperparameter in the SLIC algorithm is the K value. K determines the number of superpixels.

For convenience, it is assumed

$$K = \frac{H \times W}{T} = \frac{N}{T} \tag{16}$$

where H is the height of the HIS data, W is the width of the HIS data, N is the total number of pixels, and T is the hyperparameter we must set in all the experiments obtained by bringing K into (1)

$$S = \sqrt{T} \tag{17}$$

where S is the step size between superpixels.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 13:24:16 UTC from IEEE Xplore. Restrictions apply.

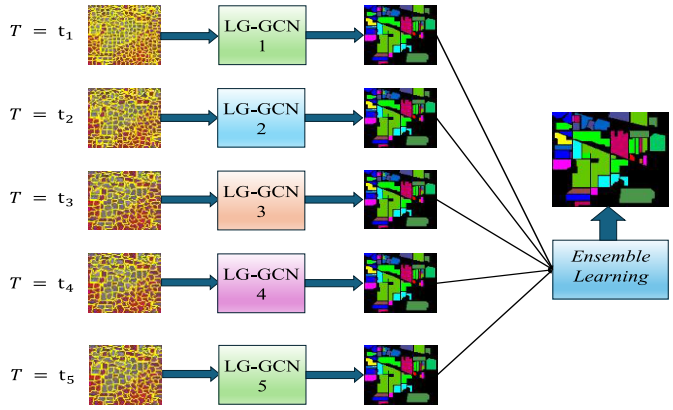


Fig. 6. Ensemble learning.

Fig. 6 illustrates the process of ensemble learning. Each model within the ensemble learning is trained independently, allowing them to learn different features and patterns from the data. This diversity aids in extracting more information from complex HSI data. Moreover, a single model may overfit due to noise in the data or sample imbalance; ensemble learning mitigates this risk by combining multiple models, thereby enhancing the model's generalization ability.

D. GCN With Local and Global Feature Fusion

The overall architecture of the LG-GCN has been shown in Fig. 4. First, LG-GCN is able to extract both spectral and spatial features of pixels when updating pixel-level graphs. This is because GCN combines the features of a multilayer perceptron (MLP) with the information passing mechanism of the graph structure. The MLP can process the spectral features of the pixels, while the information passing mechanism of the GCN allows the network to take advantage of the spatial relationships between the pixels. Therefore, LG-GCN can effectively fuse spatial–spectral features when processing HSI data. However, a pixel-level graph is only a superpixel region of the whole HSI and is not connected to other pixel-level graphs, so the extracted features are local, that is, the first step in the operation of the LG-GCN is

$$h_1 = \mathrm{GCN1}(\mathrm{Data1}) \tag{18}$$
$$h_2 = \mathrm{GCN2}(h_1) \tag{19}$$
$$h_3 = \mathrm{GCN3}(h_2) \tag{20}$$

where GCN1–3 are three different GCN layers and Data1 is the pixel-level graph data.

Second, the pixel-level spatial–spectral features are assigned to the nodes of the superpixel-level graph after summing and averaging operations, which helps to capture the features within the superpixel. Then, when updating the superpixel-level graph, the network explores higher level spatial–spectral semantics and extracts global features. Since the nodes of the superpixel-level graph are interconnected, this allows the network to understand and leverage the global information of the entire image, thus improving accuracy in performing image classification, that is, the second step in the operation of the LG-GCN is

$$H_1 = \mathrm{SGCN1}(\mathrm{Data2} + h_1) \tag{21}$$
$$H_2 = \mathrm{SGCN2}(H_1 + h_2) \tag{22}$$
$$H_3 = \mathrm{SGCN3}(H_2 + h_3) \tag{23}$$

where SGCN1–3 are three different SGCN layers and Data2 is the superpixel-level graph data. The final step is for H_3 to enter the linear classification layer for classification.

In summary, the fusion of pixel-level features (local features) with superpixel-level features (global features) can be achieved by the addition of both. The fusion helps the model to understand both the details and the overall structure of the image. When updating the features of the superpixel-level graph again, the model can better capture the semantic information in the image by extracting and fusing local–global features. Performing this fusion operation multiple times can further improve the expressive power of the features. In general, the LG-GCN model can greatly improve the accuracy of HSI classification by extracting excellent local–global and spatial–spectral features. Such comprehensive feature extraction makes the LG-GCN model perform well in processing hyperspectral data with complex spatial–spectral relationships.

III. Experiment

A. Description of HSI Datasets

1) Indian Pines (IP) Dataset: The IP dataset, collected by the AVIRIS sensor over a single landscape in Indiana, USA, consists of 224 spectral bands and is composed of 145 × 145 pixels, totaling 16 classes and 10 249 labeled pixels. After the removal of interference bands, 200 spectral bands remain. In the experiments, 450 labeled samples are selected from the labeled pixels for the training set, and 9799 samples constitute the test set. Table I on the left displays the land-cover map and the number of samples for each class.

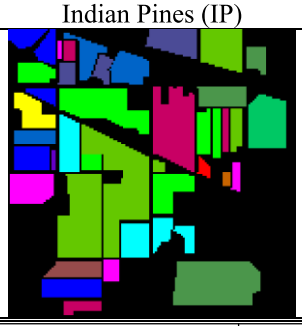
2) Pavia University (PU) Dataset: The PU dataset, captured by the ROSIS sensor above PU in Italy, has dimensions of 610 × 340 pixels, encompassing nine classes and 207 400 pixels, with 42 776 pixels labeled. The spatial resolution is 1.3 m/pixel. After discarding 12 polluted bands, the spectral dimension of PaviaU stands at 103. In the experiments, 270 labeled samples are selected from the labeled pixels for the training set, and 42 506 samples are selected from the labeled pixels for the test set. Table I in the middle shows the land-cover map and the number of samples for each class.

3) Salinas (SA) Dataset: The SA dataset, acquired via the AVIRIS sensor over the SA Valley, measures 512 × 217 pixels, with 54 129 pixels labeled across 16 classes. The spatial resolution is 3.7 m/pixel, with a wavelength range from 0.2 to 2.4 μm. After the removal of 20 nonreflective water bands, the spectral dimension of SA is 204. In the experiments, 480 labeled samples are selected from the labeled pixels for the training set, and 53 649 samples make up the test set. Table I on the right illustrates the land-cover map and the number of samples for each class.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 13:24:16 UTC from IEEE Xplore. Restrictions apply.

TABLE I

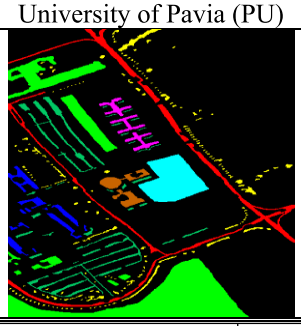
DISTRIBUTION OF LAND COVER IN THE THREE DATASETS, THE NUMBER OF TRAINING SAMPLES, AND TEST SAMPLES FOR EACH CLASS



Indian Pines (IP)

	Land-cover type	Train	Test
1	Alfalfa	30	16
2	Corn-notill	30	1398
3	Corn-mintill	30	800
4	Corn	30	207
5	Grass-pasture	30	453
6	Grass-trees	30	700
7	Grass-pasture-mowed	15	13
8	Hay-windrowed	30	448
9	Oats	15	5
10	Soybean-notill	30	942
11	Soybean-mintill	30	2425
12	Soybean-clean	30	563
13	Wheat	30	175
14	Woods	30	1235
15	Buildings-Grass-Trees-Drives	30	356
16	Stone-Steel-Towers	30	63

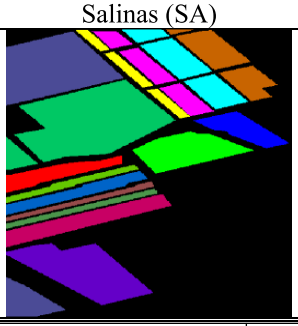
Total number of training samples: 450
Total number of testing samples: 9799



University of Pavia (PU)

	Land-cover type	Train	Test
1	Asphalt	30	6601
2	Meadows	30	18619
3	Gravel	30	2069
4	Trees	30	3034
5	Painted metal sheets	30	1315
6	Bare Soil	30	4999
7	Bitumen	30	1300
8	Self-Blocking Bricks	30	3652
9	Shadows	30	917

Total number of training samples: 270
Total number of testing samples: 42506



Salinas (SA)

	Land-cover type	Train	Test
1	Brocoli_green_weeds_1	30	1979
2	Brocoli_green_weeds_2	30	3696
3	Fallow	30	1946
4	Fallow_rough_plow	30	1364
5	Fallow_smooth	30	2648
6	Stubble	30	3929
7	Celery	30	3549
8	Grapes_untrained	30	11241
9	Soil_vinyard_develop	30	6173
10	Corn_senesced_green_weeds	30	3248
11	Lettuce_romaine_4wk	30	1038
12	Lettuce_romaine_5wk	30	1897
13	Lettuce_romaine_6wk	30	886
14	Lettuce_romaine_7wk	30	1040
15	Vinyard_untrained	30	7238
16	Vinyard_vertical_trellis	30	1777

Total number of training samples: 480
Total number of testing samples: 53649

B. Content and Configuration of the Experiment

In this study, all experiments are conducted on three publicly available hyperspectral image datasets: IP, PU, and SA. The LG-GCN model is compared with state-of-the-art CNN-, ViT-, and GCN-based methods. The CNN-based models are SSRN [17] and HybridSN [18]. The ViT-based models are SSTN [23] and CTMixer [24]. The GCN-based models are DMSGer [34], GTFN [36], CEGCN [33], and GiGCN [35]. To construct the training set, 30 labeled samples are randomly selected from each class. If a class has fewer than 30 samples, 15 labeled samples are chosen. The remaining samples construct the test set. To ensure fair evaluation, random sampling is repeated ten times, and the average accuracy (AA) is recorded, including standard deviation. Model performance is assessed using overall accuracy (OA), classification AA, and Kappa. In addition to the comparison with other methods, the detailed parts of the LG-GCN network are explored. During training, the Adam optimizer with a learning rate of 0.001 is used for 250 iterations. All experiments were run in a PyTorch environment using a CPU i7-12650H, 32-GB DDR5 RAM, and an NVIDIA GeForce RTX 4060 8G GPU.

C. Hyperparameter and Ablation Analysis

The training process of LG-GCN model starts with segmentation of HSI using the SLIC algorithm. The segmented data are then converted into graph-structured data as input for training the LG-GCN model. On the IP and PU datasets, T is set to [4], [9], [16], [25], [36] corresponding to $S =$ [2], [3], [4], [5], [6]. On the SV dataset, T is set to [16, 25, 36, 49, 64, 81], which corresponds to $S =$ [4], [5], [6], [7], [8].

TABLE II

EFFECT OF m ON THE LG-GCN MODEL

Dataset	m	1	2	3	4
IP	OA	94.84±0.89	**95.62±0.39**	94.57±0.80	94.30±0.80
	AA	97.20±0.34	**97.62±0.18**	97.21±0.30	97.06±0.32
	Kappa	94.14±1.00	**95.02±0.44**	93.83±0.90	93.53±0.90
PU	OA	98.37±0.49	**98.66±0.51**	98.10±0.52	97.68±0.46
	AA	98.22±0.36	**98.60±0.28**	97.52±0.48	96.94±0.49
	Kappa	97.85±0.64	**98.23±0.67**	97.48±0.69	96.93±0.60
SA	OA	99.21±0.15	**99.57±0.15**	99.44±0.18	99.37±0.19
	AA	99.32±0.15	**99.62±0.15**	99.42±0.19	99.37±0.22
	Kappa	99.12±0.17	**99.52±0.17**	99.38±0.20	99.30±0.21

By adjusting the value of T in (17), we can control the step size S between superpixels to obtain different graph-structured data and different LG-GCN models. Finally, the ensemble learning method with maximum voting is used to determine the classification results (see Fig. 6 for the ensemble learning process). In our experiments, we investigate the importance of the hyperparameter m in the LG-GCN model, which determines the order of neighboring nodes considered by the SGCN in the feature aggregation process. In addition, we investigated the importance of each component of the network.

1) Hyperparameter (m) in SGCN: Table II shows the effect of m on the performance of the LG-GCN model on three different datasets. On the three different datasets, OA reaches its maximum value at $m = 2$ on all three datasets and at $m = 2$ on the PU dataset. In addition, SGCN ($m = 2$) is superior to GCN here (note: SGCN is equivalent to GCN when $m = 1$).

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 13:24:16 UTC from IEEE Xplore. Restrictions apply.

TABLE III
EFFECT OF T ON THE LG-GCN MODEL

T	**4**	9	16	25	36	49	**Ensemble**
IP	**94.66±1.04**	94.09±0.51	93.16±0.70	91.73±0.93	91.78±1.01	90.53±0.71	**95.62±0.39**
PU	**97.99±0.55**	97.77±0.66	97.66±0.89	96.97±0.70	95.99±0.80	94.81±0.84	**98.66±0.51**
T	16	25	**36**	49	64	81	**Ensemble**
SA	98.52±0.50	98.66±0.34	**99.01±0.21**	98.64±0.30	98.60±0.39	98.59±0.37	**99.57±0.15**

TABLE IV
CONTRIBUTION OF BACKBONE COMPONENTS

Case	GCN Branch	SGCN Branch	Feature Fuse	Metric	IP	PU	SA
1	√	×	×	OA	91.96±0.78	95.29±0.97	95.82±0.36
				AA	95.70±0.34	95.43±0.76	97.65±0.23
				Kappa	90.88±0.88	93.77±1.26	95.35±0.40
2	×	√	×	OA	94.01±0.80	95.55±1.22	99.30±0.19
				AA	96.88±0.25	93.62±0.82	99.22±0.24
				Kappa	93.20±0.90	94.11±1.59	99.22±0.21
3	√	√	×	OA	94.97±0.71	98.32±0.38	99.25±0.41
				AA	97.31±0.22	97.96±0.39	99.37±0.22
				Kappa	94.29±0.79	97.77±0.50	99.16±0.46
4	√	√	√	OA	**95.62±0.39**	**98.66±0.51**	**99.57±0.15**
				AA	**97.62±0.18**	**98.60±0.28**	**99.62±0.15**
				Kappa	**95.02±0.44**	**98.23±0.67**	**99.52±0.17**

This is because there is a high probability that adjacent hyperpixels belong to the same label, and aggregating the information of the farther neighbors will result in the feature representations of adjacent nodes becoming similar. However, setting the m-value too large will result in generating a larger range of similarity in feature representations, making the boundaries between classes blurred, which is detrimental to classification. Therefore, in all subsequent experiments, we set m to 2.

2) T and Ensemble Learning: Table III shows the performance of the LG-GCN model with different T-values and integrated learning. On the IP and PU datasets, the LG-GCN model performs better at $T = 4$, and on the SA dataset, the LG-GCN model performs better at $T = 36$. The ensemble learning method is used on all three datasets, the performance is greatly improved, and the standard deviation becomes smaller (improved robustness).

3) Ablation Analysis: Fig. 4 shows the LG-GCN model. According to Fig. 4, the model skeleton can be divided into three pieces, which are the GCN branch, the SGCN branch, and the feature fusion operation. The GCN branch is used to process the pixel-level graph data to obtain the local spatial–spectral features, the SGCN branch is used to process the superpixel-level graph data to obtain the global spatial–spectral features, and the feature fusion operation is the key to realize the local–global feature fusion. Table IV shows the different performances of these three branches.

By observing the performance of Case-1 and Case-2, it is not difficult to find that the feature representation of the SGCN branch is much better than that of the GCN branch, which is more conducive to distinguishing different classes. In the LG-GCN model, the SGCN branch is the main part, which can mine the relationship between the superpixel points, but its disadvantage is that it cannot extract the local features inside the superpixel points, so the primary purpose of the GCN branch is to make up for this disadvantage. To prove this, Case-3 can be viewed. Case-3 uses both GCN and SGCN branches, and then, in the last step, the feature representations of both branches are summed up before doing the linear classification. Compared to Case-1 and Case-2, the performance of Case-3 is greatly improved.

However, Case-3 only conducts feature fusion at the end only, and this fusion is not high enough. Therefore, Case-4 version of LG-GCN model is proposed. The feature representations of GCN branch and SGCN branch will be operated by multilevel summation to achieve close fusion. Compared with Case-3, Case-4 has a higher level of performance.

D. Comparative Experiment

For the convenience of the discussion that follows, a brief description of the methods that have emerged is given. CNN-based methods are the SSRN and HybridSN models, both of which consist entirely of convolution. ViT-based methods are the SSTN and CTMixer models, all of which use convolution to complement ViT to get better features. GCN-based methods are the DMSGer, GTFN, GiGCN, and CEGCN models, where DMSGer and GiGCN are all composed of graph convolution and the GTFN model is a fusion of ViT and GCN.

1) IP Dataset: Table V shows the performance comparison between the LG-GCN model using ensemble learning and other models on the IP dataset. These include CNN, ViT, and GCN-based methods. The IP dataset consists of 16 classes, and the LG-GCN method leads in seven of them. In addition, the LG-GCN model outperforms the other methods in OA, AA, and Kappa. Without ensemble learning, the best performance of the LG-GCN model on the IP dataset is OA $= 94.66 \pm 1.04$ (see Table III). Even without ensemble learning, the LG-GCN model has outperformed the other methods.

Moreover, the SSRN, SSTN, and CTMixer models outperform the DMSGser and GiGCN models. This indicates that although graph convolution can theoretically extract local–global features and spatial–spectral features for high-precision HSI classification, existing GCN models that only use graph convolution underperform some methods based on CNN and ViT. The main reason is that although they recognize the importance of pixel-level and superpixel-level features, they do not effectively fuse these two types of features. Besides, in all the GCNs-based models, the GTFN model and the CEGCN model achieved the best results except for the LG-GCN model. However, the GTFN model combines ViT and GCN to improve the performance, thus significantly increasing the training time, and the CEGCN model combines convolution.

2) PU Dataset: Table VI shows the performance comparison of the LG-GCN model using ensemble learning with other models on the PU dataset. These methods include those based on CNN, ViT, and GCN. The PU dataset consists of nine classes. The LG-GCN model outperforms the other methods in terms of OA, AA, and Kappa. Without ensemble learning, the best performance of the LG-GCN model on the PU dataset is OA $= 97.99 \pm 0.55$ (see Table III). Even without ensemble learning, the LG-GCN model outperforms other methods. The CEGCN model is a close second.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 13:24:16 UTC from IEEE Xplore. Restrictions apply.

TABLE V
CLASSIFICATION RESULTS ON IP DATASETS

Class	CNN-based Methods		ViT-based Methods		Grap-based Methods				
	SSRN	HybridSN	SSTN	CTMixer	DMSGer	GTFN	GiGCN	CEGCN	LG-GCN
1	100.0±0.00	100.0±0.00	100.0±0.00	100.0±0.00	100.0±0.00	99.68±0.97	98.04±2.05	**100.0±0.00**	99.78±0.65
2	88.27±5.62	82.91±6.76	84.11±3.45	85.39±4.36	86.68±1.79	90.07±1.43	84.83±5.24	88.11±8.06	**91.46±2.93**
3	95.84±2.11	90.70±3.23	91.00±3.51	94.26±1.74	89.17±2.59	92.00±1.96	89.84±3.07	94.25±3.90	**96.31±3.49**
4	99.73±0.40	98.79±1.75	99.27±0.77	99.45±0.85	**100.0±0.00**	99.18±0.38	93.16±4.68	99.51±0.43	98.98±0.83
5	93.59±4.66	94.18±3.46	95.03±2.08	94.06±2.94	92.86±1.08	94.39±0.73	87.99±3.70	95.93±2.95	**97.03±2.11**
6	**99.63±0.30**	99.16±0.70	98.21±0.80	98.19±1.26	98.67±0.57	97.87±0.83	97.21±1.21	99.11±0.66	98.97±0.83
7	100.0±0.00	100.0±0.00	100.0±0.00	100.0±0.00	100.0±0.00	97.69±4.93	97.14±1.42	100.0±0.00	**100.0±0.00**
8	99.95±0.09	100.0±0.00	99.63±0.08	100.0±0.00	100.0±0.00	99.78±0.33	99.72±0.26	**100.0±0.00**	99.98±0.06
9	100.0±0.00	100.0±0.00	100.0±0.00	100.0±0.00	100.0±0.00	100.0±0.00	96.00±7.99	100.0±0.00	**100.0±0.00**
10	91.77±3.61	90.39±6.37	92.85±3.81	90.72±3.00	**93.61±1.35**	93.59±2.85	82.86±4.99	92.96±2.98	93.20±3.03
11	87.44±3.69	83.80±4.64	89.74±3.54	88.67±2.59	85.71±1.02	90.66±1.87	92.38±1.91	91.83±2.60	**94.01±2.11**
12	92.65±2.70	88.03±4.05	92.77±2.90	92.45±3.18	91.29±0.76	90.43±1.26	87.26±6.84	94.66±6.07	**99.65±0.43**
13	99.74±0.28	100.0±0.00	100.0±0.00	100.0±0.00	99.88±0.22	99.09±1.31	99.51±0.00	**100.0±0.00**	99.96±0.08
14	98.44±0.99	97.59±2.31	97.40±1.09	98.32±1.23	97.03±1.45	99.64±1.21	**99.66±0.38**	98.08±1.58	99.53±0.80
15	97.44±3.15	95.97±3.75	99.39±0.40	99.57±0.24	**99.86±0.33**	98.43±0.62	97.51±1.41	98.59±1.77	99.78±0.64
16	99.64±0.66	99.20±1.21	99.73±0.59	100.0±0.00	100.0±0.00	98.89±1.02	100.0±0.00	**100.0±0.00**	99.78±0.64
OA	93.04±1.24	90.37±0.88	92.63±1.25	92.67±0.85	91.51±0.64	93.89±0.43	91.77±0.76	94.27±1.89	**95.62±0.39**
AA	96.51±0.68	95.05±0.60	96.22±0.67	96.32±0.44	95.74±0.32	96.34±0.47	93.94±0.76	97.06±0.98	**97.62±0.18**
KAPPA	92.06±1.41	89.05±0.99	91.59±1.42	91.64±0.96	90.35±0.72	93.02±0.49	90.65±0.85	93.44±2.16	**95.02±0.44**

TABLE VI
CLASSIFICATION RESULTS ON PU DATASETS

Class	CNN-based Methods		ViT-based Methods		Grap-based Methods				
	SSRN	HybridSN	SSTN	CTMixer	DMSGer	GTFN	GiGCN	CEGCN	LG-GCN
1	93.90±4.46	96.56±1.06	90.94±4.40	95.15±1.18	83.20±3.94	90.86±2.69	97.14±2.93	**98.73±1.07**	98.64±1.26
2	95.76±3.67	98.46±1.42	97.17±2.14	97.64±1.05	96.18±1.23	95.93±2.16	**99.38±0.33**	96.35±4.01	98.75±0.74
3	88.62±2.57	86.81±3.42	91.50±1.69	93.67±2.29	98.16±2.00	85.65±1.81	**99.97±0.02**	99.26±0.82	99.61±0.16
4	96.60±1.35	94.54±1.94	94.72±1.01	95.43±1.79	91.52±2.55	95.77±0.93	86.57±2.47	**97.80±0.28**	94.38±1.72
5	99.99±0.02	99.52±0.42	99.56±0.19	99.78±0.13	99.68±0.21	99.67±0.20	100.0±0.00	**100.0±0.00**	99.95±0.08
6	97.91±1.74	97.28±2.29	98.93±1.11	98.74±1.47	**99.97±0.05**	98.19±1.26	99.93±0.12	99.95±0.07	99.83±0.27
7	99.80±0.35	99.43±0.60	99.84±0.22	99.97±0.06	99.83±0.27	99.17±0.54	99.90±0.12	**100.0±0.00**	99.90±0.09
8	92.84±2.43	91.16±4.36	92.13±2.74	92.36±2.39	97.55±1.50	99.01±2.47	97.90±1.11	**99.35±0.57**	99.10±0.56
9	99.49±0.32	99.06±0.89	97.52±1.14	98.55±0.71	98.00±1.01	98.23±0.69	86.50±5.18	**99.91±0.04**	97.24±1.59
OA	95.52±1.58	96.63±0.97	95.69±0.18	96.74±0.47	94.75±0.83	94.65±0.80	97.83±0.45	97.95±1.72	**98.66±0.51**
AA	96.10±0.43	95.87±0.73	95.81±0.49	96.81±0.57	96.01±0.53	94.83±0.18	96.36±0.53	**99.04±0.42**	98.60±0.28
KAPPA	94.11±2.04	95.53±1.28	94.31±0.24	95.69±0.62	93.11±1.07	92.96±1.02	97.13±0.60	97.31±2.23	**98.23±0.67**

On IP datasets where the number of class samples is not large enough (see Table I), both CNN- and ViT-based methods outperform GiGCN methods (see Table V), that is, for a small number of class samples, GCN-based methods are generally inferior to CNN- and ViT-based methods. This is because, with a small number of samples, it is difficult for these samples to establish a good graph structure relationship, so it is difficult for GCN to be able to fully mine the relationship between these samples. On the other hand, CNN and ViT only need to learn the features of the samples and can still learn and extract features more efficiently when dealing with small-sample data, resulting in higher accuracy. However, on the PU dataset, the GiGCN model can fully maximize its advantages due to the large number of samples in each class. (i.e., each class has enough samples to generate good graph structure relations). However, the performance of the DMSGer model on the PU dataset is still poor, probably because it does not learn a good feature representation. The worst performer among all models is the GTFN model. Although the GTFN model is the only one of the GNN-based methods that combines ViT, it does not show the expected high performance on the PU dataset (compared to the other methods in Table VI). Both CNN- and ViT-based methods performed mediocrely.

3) SA Dataset: Table VII shows the performance comparison of the LG-GCN model using ensemble learning on the SA dataset with other models. These include methods based on CNN, ViT, and GCN. The SA dataset consists of 16 classes, and the LG-GCN method leads in nine of them. The LG-GCN model outperforms the other methods in OA, AA, and Kappa. Without ensemble learning, the best performance of the LG-GCN model on the SA dataset is OA = 99.01±0.21 (see Table III). Even without ensemble learning, the LG-GCN model outperforms the other methods.

As can be seen from Table I, the sample distribution of different classes in the SA dataset is relatively concentrated, and the number of samples in each class is also relatively sufficient. This concentrated and sufficient sample distribution is conducive to constructing a favorable graph

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 13:24:16 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
CLASSIFICATION RESULTS ON SA DATASETS

Class	CNN-based Methods		ViT-based Methods		Grap-based Methods				
	SSRN	HybridSN	SSTN	CTMixer	DMSGer	GTFN	GiGCN	CEGCN	LG-GCN
1	100.0±0.00	100.00±0.00	99.99±0.01	99.98±0.03	99.95±0.07	100.0±0.00	100.0±0.00	100.0±0.00	**100.0±0.00**
2	100.0±0.00	100.0±0.00	99.97±0.02	99.96±0.07	99.68±0.31	99.94±0.07	100.0±0.00	100.0±0.00	**100.0±0.00**
3	100.0±0.00	100.0±0.00	100.0±0.00	99.96±0.05	100.0±0.00	99.80±0.24	100.0±0.00	100.0±0.00	**100.0±0.00**
4	99.85±0.22	99.91±0.07	99.71±0.23	99.62±0.65	99.01±0.66	99.71±0.19	**100.0±0.00**	**99.98**±0.02	99.46±1.52
5	**99.53±0.37**	98.92±0.74	99.22±0.65	99.07±0.86	98.23±0.60	96.41±1.15	98.02±0.16	98.70±1.03	98.87±0.41
6	99.43±0.06	99.97±0.04	99.90±0.13	**99.97±0.03**	99.29±0.65	99.66±0.36	99.92±0.00	100.0±0.00	99.74±0.14
7	99.96±0.03	99.93±0.10	99.85±0.32	99.96±0.09	99.87±0.10	99.72±0.22	99.96±0.02	99.94±0.06	**99.97±0.09**
8	90.84±1.92	92.31±2.49	90.60±3.40	93.71±3.44	95.23±3.18	91.64±0.76	96.87±2.41	92.47±2.33	**99.37±0.48**
9	99.99±0.00	99.99±0.02	99.93±0.08	100.0±0.00	99.53±0.47	99.78±0.26	100.0±0.00	100.0±0.00	**100.0±0.00**
10	98.85±0.81	98.46±1.23	96.83±2.15	98.31±1.30	**99.29±0.69**	95.34±0.71	97.21±1.79	98.57±0.30	98.70±1.20
11	99.95±0.12	99.95±0.10	99.90±0.09	99.96±0.06	98.37±0.61	99.79±0.38	99.97±0.04	100.0±0.00	**100.0±0.00**
12	99.95±0.08	99.76±0.51	99.92±0.07	99.81±0.26	96.54±1.04	99.28±0.49	99.68±0.38	100.0±0.00	99.61±0.39
13	**99.97±0.04**	99.92±0.19	99.88±0.16	99.92±0.09	99.68±0.49	99.82±0.15	99.34±0.00	99.77±0.34	99.86±0.11
14	**99.85±0.25**	99.35±0.97	99.87±0.13	99.71±0.40	98.41±1.86	98.92±0.84	97.91±2.20	99.49±0.27	99.08±0.68
15	94.86±1.83	94.14±1.75	97.96±0.69	95.56±2.20	92.21±3.88	94.11±1.82	96.26±1.20	96.24±1.45	**99.30±0.36**
16	99.42±0.51	99.32±0.88	99.77±0.25	99.53±0.41	99.92±0.11	98.82±0.45	99.34±0.90	99.21±0.83	**99.96±0.11**
OA	97.26±0.45	97.39±0.65	97.47±0.69	97.88±0.72	97.46±1.03	96.81±0.28	98.48±0.60	97.72±0.57	**99.57±0.15**
AA	98.93±0.17	98.87±0.24	98.96±0.18	99.07±0.21	98.45±0.48	98.30±0.15	99.03±0.24	99.02±0.21	**99.62±0.15**
KAPPA	96.95±0.50	97.10±0.72	97.19±0.76	97.64±0.80	97.17±1.15	96.45±0.31	98.31±0.67	97.46±0.64	**99.52±0.17**

structure relationship, which enables the GCN model to extract the features of the samples more effectively. As shown in Table VII, except for the GTFN model, the other GCN-based models are either about the same as or better than the CNN- and ViT-based models.

4) Small Summary: The patch-based method crops many $L \times L$ size patches centered on the labeled pixel points, and these patches are trainable data. The patch-based model is restricted to the inside of the patch, and the information outside the patch cannot be attended to. Even if the global features are extracted using ViT, such global features are only the global features of the patch, not the global features of the whole image. For example, the GTFN model performs well on the IP dataset, but it does not perform well on the PU and SA datasets. There are two reasons for this. First, it is a single-branch model, the front graph convolution layer is equivalent to a low-level feature extractor, and the high-level features are extracted by the ViT at the back, which leads to the advantage of graph convolution not being fully exploited. Second, it is patch-based, the graph convolution can only be confined within a fixed-size patch, and it cannot extract the global features of the whole HSI. Both CNN- and ViT-based models in this experiment are patch-based methods. They also have the same problems as the GTFN model.

By contrast, the superpixel-based methods generally use the SLIC segmentation algorithm to determine the superpixel points first and then form the graph structure data (i.e., the trainable data). The superpixel-based model can extract not only local features but also global features of the entire image. For example, the CEGCN model is a two-branch model, the top branch is the graph convolutional layer to extract global superpixel-level features, the bottom branch is the convolution layer to extract local pixel-level features, and finally, the two branches are merged into one branch. Because it combines pixel-level features and superpixel-level features, this makes it perform well on the IP and PU datasets. In addition to the CEGCN model, the GiGCN and LG-GCN models are all superpixel-based methods. The most unique aspect of the LG-GCN model is the feature fusion part of LG-GCN. On all three datasets, the LG-GCN model is in the absolute lead, which proves that the feature fusion part is important.

Both GiGCN and CEGCN are two-branch network frameworks designed to extract pixel- and superpixel-level features. The top and bottom branches of the GiGCN model are graph convolutions. On the IP dataset, the GiGCN model does not perform as well as the CEGCN model, but on the PU dataset, both perform equally well. However, on the SA dataset, the GiGCN model outperforms the CEGCN model. This suggests that while the CEGCN model combined with convolution performs well on the IP dataset, which has an insufficient number of class samples, the GiGCN model performs better on the SA dataset, which has a sufficient number of class samples. Furthermore, from Table I, it can be obtained that class "11" has the most pixel points on the IP dataset, class "2" on the PU dataset, and class "8" on the SA dataset. The accuracy of the LG-GCN or GiGCN model on the class above this is well ahead of the others. This reflects the fact that graph convolution performs best on datasets that can generate good graph structure relationships and on classes with the highest number of pixels. This is because a large number of samples mean that a more complex topology can be constructed for graph convolution to work.

E. Visualization of Classification Results

Figs. 7–9 show the visualized classification results of each method on the IP, PU, and SA datasets, respectively. However, it is difficult to observe significant performance differences because the difference between the highest and lowest accuracy is not more than 5%. For the convenience of discussion, we equate a color to a class.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 13:24:16 UTC from IEEE Xplore. Restrictions apply.

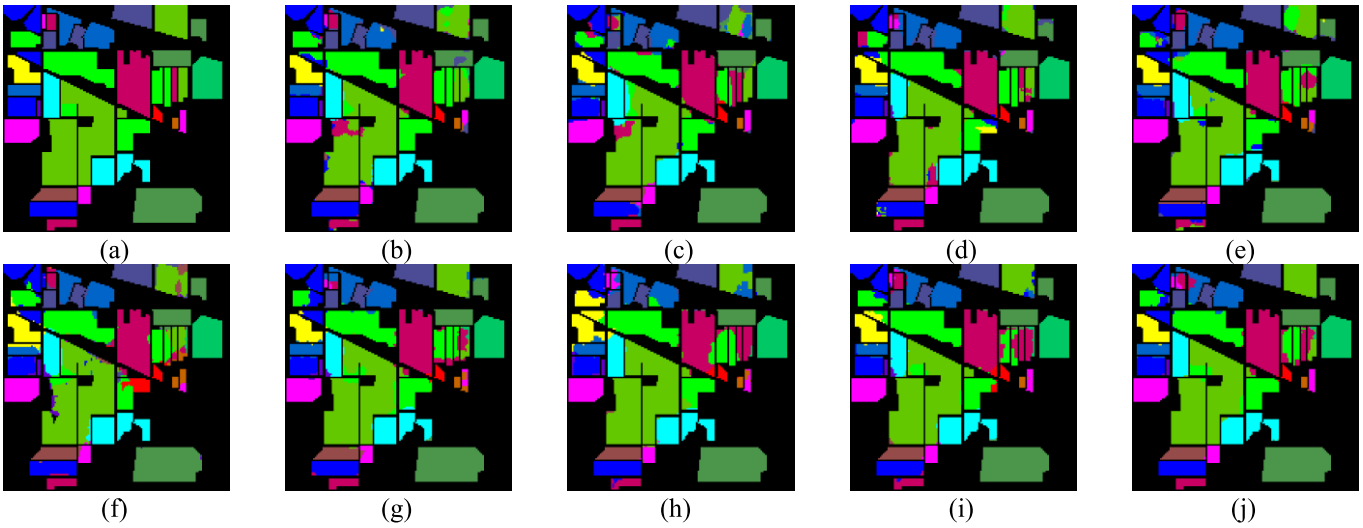


Fig. 7. Classification graph of the IP dataset. (a) Ground truth. (b) 3D-CNN. (c) HybridSN. (d) SSTN. (e) CTMixer. (f) DMSGer. (g) GTFN. (h) GiGCN. (i) CEGCN. (j) LG-GCN.

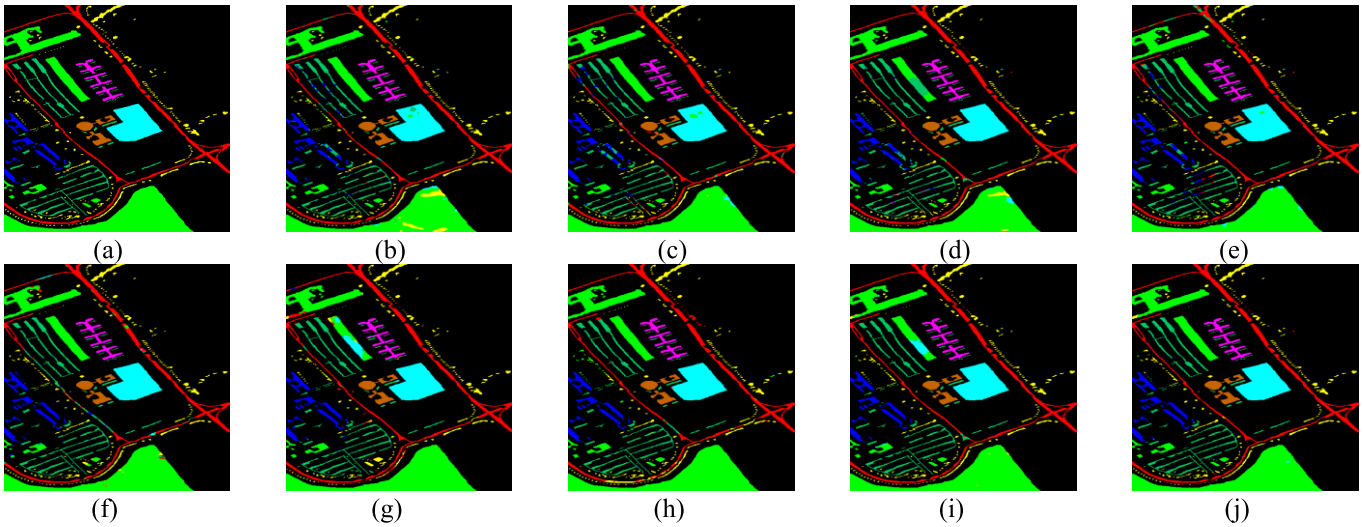


Fig. 8. Classification graph of the PU dataset. (a) Ground truth. (b) 3D-CNN. (c) HybridSN. (d) SSTN. (e) CTMixer. (f) DMSGer. (g) GTFN. (h) GiGCN. (i) CEGCN. (j) LG-GCN.

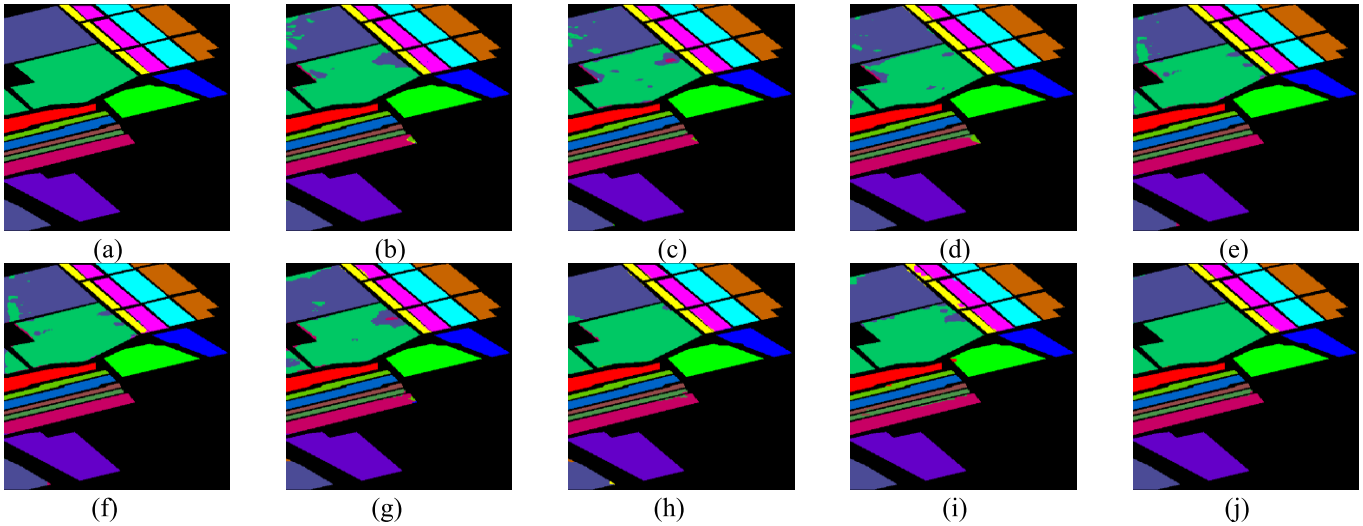


Fig. 9. Classification graph of the SA dataset. (a) Ground truth. (b) 3D-CNN. (c) HybridSN. (d) SSTN. (e) CTMixer. (f) DMSGer. (g) GTFN. (h) GiGCN. (i) CEGCN. (j) LG-GCN.

The methods from Figs. 7(b)–(f) to 9(b)–(f) are patch-based methods with the limitation that they can only focus on the information within the patch and cannot capture the global information of the image. A drawback of this method is that

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 13:24:16 UTC from IEEE Xplore. Restrictions apply.

TABLE VIII
CLASSIFICATION RESULTS UNDER NOISE

Dataset	IP			PU			SA		
Standard Deviation	0	10	100	0	10	100	0	10	100
SSRN	93.04±1.24	92.60±1.37	90.48±0.75	95.52±1.58	95.79±0.36	93.13±1.68	97.26±0.45	96.27±0.84	89.62±1.44
HybridSN	90.37±0.88	92.15±0.88	92.07±0.92	96.63±0.97	96.05±0.63	93.37±0.99	97.39±0.65	98.07±0.30	96.40±0.73
SSTN	92.63±1.25	88.74±0.42	86.64±1.25	95.69±0.18	90.94±1.48	89.04±2.33	97.47±0.69	96.04±0.87	94.78±0.85
CTMixer	92.67±0.85	94.11±1.48	93.27±0.48	96.74±0.47	95.06±1.13	92.91±1.70	97.88±0.72	97.68±0.65	96.58±0.68
GTFN	93.89±0.43	78.75±0.15	79.90±0.36	94.65±0.80	48.38±1.02	30.05±0.45	96.81±0.28	57.20±0.70	37.74±0.32
CEGCN	94.27±1.89	93.39±1.04	87.08±1.30	97.95±1.72	97.85±1.79	98.03±1.67	97.72±0.57	97.33±0.87	92.15±3.20
DMSGer	91.51±0.64	92.06±0.95	91.74±0.18	94.75±0.83	95.46±0.23	94.15±0.72	97.46±1.03	97.58±0.20	96.31±0.76
GiGCN	91.77±0.76	90.72±1.81	90.87±1.84	97.83±0.45	97.93±0.34	97.89±0.40	98.48±0.60	97.88±0.40	98.65±0.46
LG-GCN	**95.62±0.39**	**95.49±0.81**	**95.45±0.66**	**98.66±0.28**	**98.49±0.64**	**98.58±0.50**	**99.57±0.15**	**99.39±0.26**	**99.41±0.20**

if a pixel of a certain color is misclassified, the wrong color it is assigned tends to be random rather than its neighboring colors. This feature is particularly evident in Fig. 7, indicating that these patch-based methods overly focus on the features of patches and ignore the overall image features. The GTFN model suffers from the same problem.

However, in Figs. 7–9, even if our LG-GCN model predicts incorrectly, the incorrect colors appear only at the edges and are the same as the adjacent colors. This is because the LG-GCN model not only focuses on the features in the local region (i.e., pixel-level features) but also focuses on the features in the global region (i.e., superpixel-level features). The above case is also true for the CEGCN and GiGCN models. Although the DMSGer model exists to be assigned error colors that are random, this is because the feature representation is not very good. After all, in all methods, the DMSGer model performs poorly on all three datasets.

F. Noise Test

In order to explore the performance of LG-GCN models under noise interference, we conducted noise test experiments on all models. In this experiment, we added Gaussian noise to the original hyperspectral images by generating Gaussian noise with a mean of 0 and standard deviations of 0, 10, and 100. Since all HSIs with added noise undergo a batchnorm operation before entering the model, even if the noise value is large, the interference of the noise will be reduced after batchnorm. The final experimental results are shown in Table VIII.

Among all the models in Table VIII, only the DMSGer, GiGCN, and LG-GCN models, located at the bottom, are composed entirely of graph convolution. As can be seen in Table VIII, these three models are insensitive to noise regardless of the dataset, and the effect of noise on their performance is minimal. The other models either contain a CNN component or a ViT component, and they are relatively sensitive to noise. In addition, the ensemble learning strategy is theoretically resistant to noise. Both the LG-GCN model and the GiGCN model use an ensemble learning strategy. However, compared with the DMSGer model that did not use ensemble learning, it can be concluded that the ensemble learning method does not give much contribution to the model's anti-noise ability, and the anti-noise ability is still mainly the graph convolutional neural in action.

In previous experiments, we concluded that GCNs consisting entirely of graph convolutions perform best on datasets with sufficiently complex graph structural relationships. Noise has less impact on the establishment of graph structure relationships and instead has more impact on the vector values of superpixel nodes and pixel nodes. However, graph convolution can remove noise and enhance robustness by aggregating information from node neighborhoods to update node representations. In addition, graph convolution can better capture the global relationships in the data by exploiting the topological information in the graph structure, thus enhancing the robustness to noise. Therefore, as long as the graph structure relationships are well formed, the performance of a GCN consisting entirely of graph convolution does not change significantly. In contrast, CNNs and ViTs may amplify noise when dealing with local features. This is because CNNs and ViTs rely on data to learn feature representations during training. If the training data contain noise, the model also learns these noisy features, thus amplifying noise during testing and training.

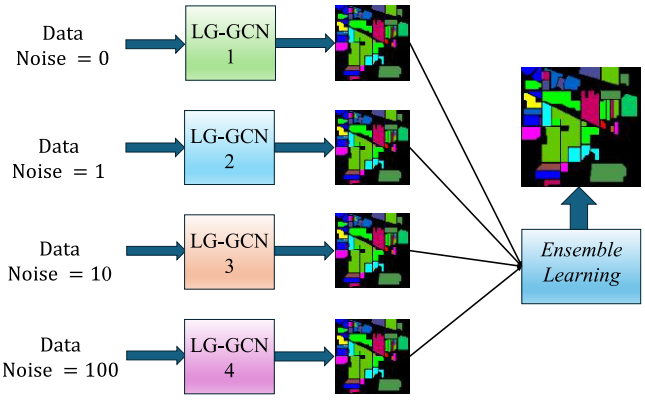


Fig. 10. Ensemble learning of LG-GCN models with different noises.

TABLE IX
ENSEMBLE LEARNING WITH DIFFERENT NOISES

Dataset	NOISE	0	1	10	100	Ensemble
IP	OA	95.62±0.39	95.56±0.68	95.49±0.81	95.45±0.66	**95.66±0.66**
	AA	97.62±0.18	97.53±0.41	97.46±0.47	97.46±0.46	**97.62±0.43**
	Kappa	95.02±0.44	94.95±0.77	94.87±0.92	94.82±0.75	**95.06±0.75**
PU	OA	98.66±0.51	98.55±0.48	98.49±0.64	98.58±0.50	**98.79±0.40**
	AA	98.60±0.28	98.29±0.36	98.23±0.41	98.24±0.36	**98.39±0.32**
	Kappa	98.23±0.67	98.09±0.63	98.01±0.84	98.13±0.65	**98.40±0.53**
SA	OA	99.57±0.15	99.43±0.19	99.39±0.26	99.41±0.20	**99.55±0.19**
	AA	99.62±0.15	99.44±0.26	99.42±0.26	99.42±0.25	**99.54±0.25**
	Kappa	99.52±0.17	99.37±0.21	99.32±0.29	99.34±0.23	**99.46±0.21**

We also conducted an ensemble learning experiment involving noise. The specific experimental process is shown in Fig. 10. In this experiment, different LG-GCN models were trained using Gaussian noise with a mean of 0 and standard deviations of 1, 10, and 100. Finally, these models were combined with an LG-GCN model trained without noise for ensemble learning. The experimental results are shown in Table IX. From Table IX, it can be seen that the LG-GCN model achieved certain performance improvements on the IP and PU datasets, but ensemble learning had no effect on the SA dataset. On the IP and PU datasets, different noise levels allowed the LG-GCN model to learn different

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 13:24:16 UTC from IEEE Xplore. Restrictions apply.

TABLE X
COMPARISON OF RUNNING TIME

Methods	Runing time								
	IP			PU			SA		
	Train	Test	Total	Train	Test	Total	Train	Test	Total
SSRN	186.22s	270.42s	456.64s	164.36s	569.91s	734.27s	158.55s	568.48s	727.03
HybridSN	166.68s	237.10s	403.78s	179.36s	591.03s	770.39	303.33s	1010.11s	1313.44s
SSTN	186.61s	339.99s	526.60s	167.88s	697.71s	865.59s	192.64s	1004.63s	1197.27s
CTMixer	172.05s	262.13s	434.18s	166.35s	672.44s	838.79s	176.21s	801.60s	977.81s
DMSGer	28.24s	57.50s	85.74s	84.56s	224.21s	308.77s	139.16s	303.52	442.68s
GTFN	190.58s	63.41s	253.99s	136.46s	259.00s	395.46s	213.95s	329.25	543.20s
GiGCN	6.81s	6.21s	13.02s	35.47s	32.00s	67.47s	22.37s	21.32s	43.69s
CEGCN	11.42s	0.34s	11.76s	91.99s	3.48s	95.47s	44.02s	1.64s	46.06s
LG-GCN	8.21s	9.01s	17.22s	29.47s	20.67s	50.14s	19.69s	15.71s	35.40s

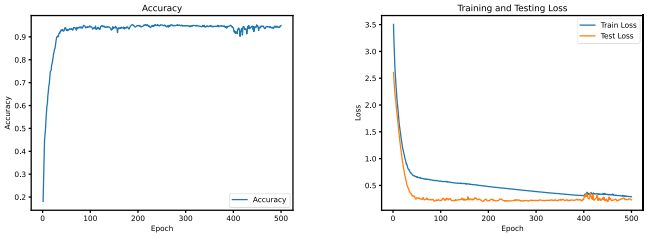


Fig. 11. Curves of accuracy and loss variation during training on the IP dataset.

feature representations, thereby improving accuracy after ensemble learning. However, the SA dataset has concentrated spatial distribution and balanced class numbers, with prominent spatial features. Despite the different noise levels, the LG-GCN model learned similar feature representations on the SA dataset (i.e., it is easier to learn distinguishable spatial feature representations), so ensemble learning did not perform well on the SA dataset.

G. Comparison of Computational Efficiency

In addition to the accuracy performance, computational efficiency experiments were also conducted. Table X shows the runtimes of the different methods on the three public datasets. The runtime includes training and testing time. The whole training process is to first update the model parameters by backpropagation, then evaluate the model on the test or validation set, and save the parameters if the performance improves. The training time is the time consumed to update the parameters and the testing time is the time consumed to save the best model.

Table X shows that the total running time of the LG-GCN model is very short, and the difference in running time with the GiGCN and CEGCN models is very small. In contrast, the CNN and ViT-based models have much higher runtimes due to the high number of parameters and the high computational cost of 3-D convolution. It indicates that graph convolutional models have a significant advantage in terms of runtime cost.

H. About Overfitting

Figs. 11–13 show the curves of accuracy and loss variation during training in three different datasets. As shown in these three figures, on the IP dataset, the LG-GCN model is obviously overfitted after the epoch reaches 400; on the PU dataset, it is obviously overfitted after the epoch reaches 300; and on the SA dataset, it is obviously overfitted after the epoch reaches 350. However, it can also be noticed that the convergence of the LG-GCN model is extremely fast, and it is close to convergence after 100 epochs. There is no significant overfitting before 250 epochs. In order to avoid overfitting, it is recommended to choose 200–250 epochs, so in the above experiment, the LG-GCN model chooses to use 250 epochs.

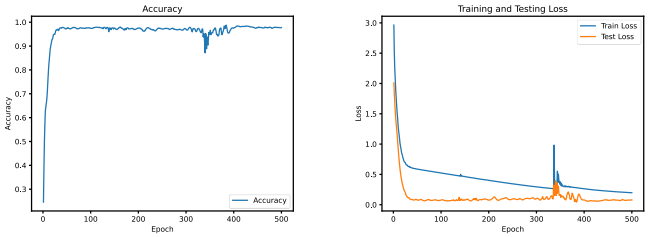


Fig. 12. Curves of accuracy and loss variation during training on the PU dataset.

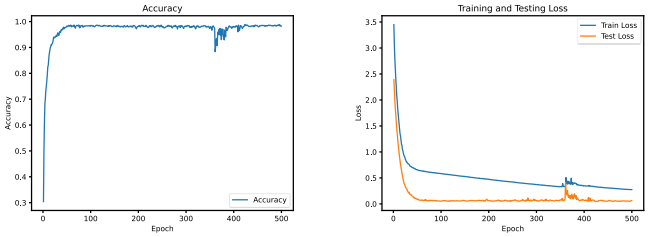


Fig. 13. Curves of accuracy and loss variation during training on the SA dataset.

However, we observed that during the training process, both the accuracy and loss curves exhibit certain fluctuations. This could be attributed to the complexity of the graph structure and the characteristics of graph convolution operations. Graph convolutional networks need to handle complex graph structures, which may contain many nodes and edges. Due to the sparsity and irregularity of graphs, gradients are prone to vanishing or exploding during backpropagation. In addition, graph convolution operations aggregate the features of a node with those of its neighboring nodes. This aggregation can cause gradients to attenuate or amplify layer by layer during propagation. These factors collectively increase the volatility during training, affecting the stability of the LG-GCN model.

IV. CONCLUSION

In this article, we propose a novel end-to-end semi-supervised network that can extract local features of pixel-level graphs and global features of superpixel-level graphs. The three parts of the LG-GCN model are the GCN branch, the SGCN branch, and the feature fusion operation; the GCN branch is used to process the pixel-level graphic data in order to extract the spectral features of the pixels and the local spatial features; and the SGCN branch is used to process the superpixel-level data in order to extract the spectral features and the global spatial features. The GCN branch is used to process pixel-level data to extract spectral and local spatial features of pixels. The feature fusion operation is a multi-level fusion of the features extracted from the two branches.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 13:24:16 UTC from IEEE Xplore. Restrictions apply.

Finally, the LG-GCN model is able to extract better local–global and spatial–spectral feature representations. To cope with the complexity and high dimensionality of HSI, the ensemble learning strategy is introduced to enhance the LG-GCN model, which not only improves the performance but also reduces the prediction bias. Also, the results show that the LG-GCN model outperforms other methods even without ensemble learning. In addition, the LG-GCN model has a short runtime and high computational efficiency. In addition, it has been proved in experiments that the local global feature fusion operation of LG-GCN is powerful.

In our experiments, we found that graph convolution performs best on datasets with sufficiently complex and clear graph structure relationships. On datasets with an insufficient number of class samples, the CEGCN model combined with convolution performs well, while on datasets with enough class samples, the GiGCN model consisting entirely of graph convolution performs better. For HSI datasets that generate excellent graph-structured relationships, graph convolution is good at capturing the complex relationships between nodes to achieve good classification results. If the dataset has terrible graph structure relationships, replacing graph convolution with convolution or ViT may achieve better results. However, on the three datasets in the experiment, graph convolutional networks with proper design (e.g., the LG-GCN model) still perform better than CNNs and ViTs and take less time.

Graph convolution updates node representations by aggregating information from the node's neighborhood, which removes noise and enhances robustness. In addition, graph convolution uses topological information in the graph structure to better capture global patterns and relationships in the data, thus improving robustness to noise. Therefore, as long as the graph structure relationships are well formed, the performance of GCN does not change significantly. Graph convolution specializes in mining relationships between different nodes.

REFERENCES

[1] M. Dalponte, H. O. Ørka, T. Gobakken, D. Gianelle, and E. Næsset, "Tree species classification in boreal forests with hyperspectral data," *IEEE Trans. Geosci. Remote Sens.*, vol. 51, no. 5, pp. 2632–2645, May 2013.

[2] C. Weber et al., "Hyperspectral imagery for environmental urban planning," in *Proc. IEEE Int. Geosci. Remote Sens. Symp.*, Jul. 2018, pp. 1628–1631.

[3] A. Bannari, A. Pacheco, K. Staenz, H. McNairn, and K. Omari, "Estimating and mapping crop residues cover on agricultural lands using hyperspectral and IKONOS data," *Remote Sens. Environ.*, vol. 104, no. 4, pp. 447–459, Oct. 2006.

[4] M. Teke, H. S. Deveci, O. Haliloglu, S. Z. Gürbüz, and U. Sakarya, "A short survey of hyperspectral remote sensing applications in agriculture," in *Proc. 6th Int. Conf. Recent Adv. Space Technol. (RAST)*, Jun. 2013, pp. 171–176.

[5] C. Jänicke, A. Okujeni, S. Cooper, M. Clark, P. Hostert, and S. van der Linden, "Brightness gradient-corrected hyperspectral image mosaics for fractional vegetation cover mapping in Northern California," *Remote Sens. Lett.*, vol. 11, no. 1, pp. 1–10, Jan. 2020.

[6] Z. Lv, M. Zhang, W. Sun, J. A. Benediktsson, T. Lei, and N. Falco, "Spatial-contextual information utilization framework for land cover change detection with hyperspectral remote sensed images," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 4411911.

[7] F. Melgani and L. Bruzzone, "Classification of hyperspectral remote sensing images with support vector machines," *IEEE Trans. Geosci. Remote Sens.*, vol. 42, no. 8, pp. 1778–1790, Aug. 2004.

[8] S. Amini, S. Homayouni, A. Safari, and A. A. Darvishsefat, "Object-based classification of hyperspectral data using random forest algorithm," *Geo-Spatial Inf. Sci.*, vol. 21, no. 2, pp. 127–138, 2018.

[9] B. Zhang, S. Li, X. Jia, L. Gao, and M. Peng, "Adaptive Markov random field approach for classification of hyperspectral imagery," *IEEE Geosci. Remote Sens. Lett.*, vol. 8, no. 5, pp. 973–977, Sep. 2011.

[10] Y. Guo, S. Han, Y. Li, C. Zhang, and Y. Bai, "K-nearest neighbor combined with guided filter for hyperspectral image classification," *Proc. Comput. Sci.*, vol. 129, pp. 159–165, Jan. 2018.

[11] Y. Chen, Z. Lin, X. Zhao, G. Wang, and Y. Gu, "Deep learning-based classification of hyperspectral data," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 7, no. 6, pp. 2094–2107, Jun. 2014.

[12] W. Hu, Y. Huang, L. Wei, F. Zhang, and H. Li, "Deep convolutional neural networks for hyperspectral image classification," *J. Sensors*, vol. 2015, pp. 1–12, Jul. 2015.

[13] V. Slavkovikj, S. Verstockt, W. De Neve, S. Van Hoecke, and R. Van de Walle, "Hyperspectral image classification with convolutional neural networks," in *Proc. 23rd ACM Int. Conf. Multimedia.*, 2015, pp. 1159–1162.

[14] Y. Li, H. Zhang, and Q. Shen, "Spectral–spatial classification of hyperspectral imagery with 3D convolutional neural network," *Remote Sens.*, vol. 9, no. 1, p. 67, Jan. 2017.

[15] A. B. Hamida, A. Benoit, P. Lambert, and C. B. Amar, "3-D deep learning approach for remote sensing image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 8, pp. 4420–4434, Aug. 2018.

[16] M. He, B. Li, and H. Chen, "Multi-scale 3D deep convolutional neural network for hyperspectral image classification," in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, Sep. 2017, pp. 3904–3908.

[17] Z. Zhong, J. Li, Z. Luo, and M. Chapman, "Spectral–spatial residual network for hyperspectral image classification: A 3-D deep learning framework," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 2, pp. 847–858, Feb. 2018.

[18] S. K. Roy, G. Krishna, S. R. Dubey, and B. B. Chaudhuri, "HybridSN: Exploring 3-D-2-D CNN feature hierarchy for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 17, no. 2, pp. 277–281, Feb. 2020.

[19] J. Feng et al., "Class-aligned and class-balancing generative domain adaptation for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5509617.

[20] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," 2020, *arXiv:2010.11929*.

[21] D. Hong et al., "SpectralFormer: Rethinking hyperspectral image classification with transformers," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5518615.

[22] L. Sun, G. Zhao, Y. Zheng, and Z. Wu, "Spectral–spatial feature tokenization transformer for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5522214.

[23] Z. Zhong, Y. Li, L. Ma, J. Li, and W.-S. Zheng, "Spectral–spatial transformer network for hyperspectral image classification: A factorized architecture search framework," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5514715.

[24] J. Zhang, Z. Meng, F. Zhao, H. Liu, and Z. Chang, "Convolution transformer mixer for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, pp. 1–5, 2022.

[25] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," 2016, *arXiv:1609.02907*.

[26] Q. Li, Z. Han, and X. Wu, "Deeper insights into graph convolutional networks for semi-supervised learning," in *Proc. AAAI Conf. Artif. Intell.*, 2018, pp. 3538–3545.

[27] A. Qin, Z. Shang, J. Tian, Y. Wang, T. Zhang, and Y. Y. Tang, "Spectral–spatial graph convolutional networks for semisupervised hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 16, no. 2, pp. 241–245, Feb. 2019.

[28] S. Wan, C. Gong, P. Zhong, B. Du, L. Zhang, and J. Yang, "Multiscale dynamic graph convolutional network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 5, pp. 3162–3177, May 2020.

[29] P. Yang, L. Tong, B. Qian, Z. Gao, J. Yu, and C. Xiao, "Hyperspectral image classification with spectral and spatial graph using inductive representation learning network," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 14, pp. 791–800, 2021.

[30] Y. Ding, Y. Chong, S. Pan, Y. Wang, and C. Nie, "Spatial–spectral unified adaptive probability graph convolutional networks for hyperspectral image classification," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 7, pp. 3650–3664, Jul. 2023.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 13:24:16 UTC from IEEE Xplore. Restrictions apply.

[31] Y. Ding et al., "Multi-scale receptive fields: Graph attention neural network for hyperspectral image classification," *Expert Syst. Appl.*, vol. 223, Aug. 2023, Art. no. 119858.
[32] R. Achanta, A. Shaji, K. Smith, A. Lucchi, P. Fua, and S. Süsstrunk, "SLIC superpixels compared to state-of-the-art superpixel methods," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 34, no. 11, pp. 2274–2282, Nov. 2012.
[33] Q. Liu, L. Xiao, J. Yang, and Z. Wei, "CNN-enhanced graph convolutional network with pixel- and superpixel-level feature fusion for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 10, pp. 8657–8671, Oct. 2021.
[34] Y. Yang et al., "Semi-supervised multiscale dynamic graph convolution network for hyperspectral image classification," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 58, no. 5, pp. 3162–3177, May 2020.
[35] S. Jia, S. Jiang, S. Zhang, M. Xu, and X. Jia, "Graph-in-graph convolutional network for hyperspectral image classification," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 1, pp. 1157–1171, Jan. 2024.
[36] A. Yang, M. Li, Y. Ding, D. Hong, Y. Lv, and Y. He, "GTFN: GCN and transformer fusion network with spatial–spectral features for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 6600115.
[37] F. Wu, A. H. Souza, T. Zhang, C. Fifty, T. Yu, and K. Q. Weinberger, "Simplifying graph convolutional networks," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2019, pp. 6861–6871.



Yufan Wang received the B.S. degree in optoelectronic information science and engineering from the Hunan University of Arts and Sciences, Changde, China, in 2021. He is currently pursuing the M.S. degree in computer science and information technology with Harbin Normal University, Harbin, China.

His research interests include machine learning and hyperspectral remote sensing image classification.



Xiaodong Yu received the B.S. and M.S. degrees in computer science and technology from Harbin Normal University, Harbin, China, in 2002 and 2007, respectively, and the Ph.D. degree in software engineering from Harbin Engineering University, Harbin, in 2020.

Since 2021, he has been a Professor with the Department of Computer Science and Technology, Harbin Normal University. His research interests include artificial intelligence and pattern recognition, computer vision, and remote sensing image processing.



Hongbin Dong received the B.S. and M.S. degrees in computer science and technology from Harbin Ship Engineering College, Harbin, China, in 1986 and 1995, respectively, and the Ph.D. degree in computer science and information technology from Beijing Jiaotong University, Beijing, China, in 2006.

Since 2005, he has been a Professor with the Department of Computer Science and Technology, Harbin Engineering University, Harbin, China. His research interests include natural computation, multiagent systems, machine learning, and data mining.

Dr. Dong is a member of the Committee on Artificial Intelligence and Pattern Recognition of the Chinese Computer Society and the Committee on Granular Computing and Knowledge Discovery of Chinese Academy of Artificial Intelligence and a Senior Member of the Computer Society.



Shuying Zang received the B.S. degree from Harbin Normal University, Harbin, China, in 1986, and the M.S. degree from Nanjing University, Nanjing, China, in 1989, and the Ph.D. degree from Northeast Forestry University, Harbin, in 1998.

Since 2000, she has been a Professor with the Department of Geographical Sciences, Harbin Normal University. Her research interests include 3S technology development and application, and remote sensing monitoring.

Dr. Zang is currently the Executive Director of the Geographical Society of China and the President of the Geographical Society of Heilongjiang Province.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 13:24:16 UTC from IEEE Xplore. Restrictions apply.